

Network Traffic Analysis Using Wireshark

A Practical Cybersecurity & Network Security Monitoring Investigation

Prepared by: **Balamohan**

Focus Areas: Networking | Cybersecurity | Packet Analysis | Security Monitoring

Environment: Windows 10 / Windows 11

Primary Tool: Wireshark (with Npcap)

Evidence File: network_capture.pcapng

This report documents the capture, filtering, and protocol-level analysis of live network traffic performed for educational, defensive, and authorized network-analysis purposes.

Table of Contents

1. Introduction
2. Objective
3. Tools Used
4. Network Environment
5. Packet Capture Method
6. ICMP Analysis
7. DNS Analysis
8. TCP Analysis
9. TLS / HTTPS Analysis
10. Endpoint Analysis
11. Conversation Analysis
12. Suspicious Traffic Analysis
13. Findings
14. Conclusion

1. Introduction

This report presents a practical cybersecurity and networking investigation titled **Network Traffic Analysis Using Wireshark**. The project focuses on capturing, filtering, and analyzing live network packets generated from a Windows-based system in order to understand how devices communicate over a network and how that communication can be examined for troubleshooting and basic security-monitoring purposes.

Wireshark, a widely used open-source packet analysis tool, was used together with Npcap to capture traffic at the network interface level. The investigation covers multiple protocol layers — from Ethernet and IP addressing up through DNS, ICMP, TCP, UDP, and TLS/HTTPS — and applies Wireshark’s statistical views (Endpoints and Conversations) to build a complete picture of host communication.

The analysis follows a structured, evidence-based workflow: generate representative traffic, capture it, apply targeted display filters, inspect individual packets, correlate findings with native Windows command-line tools, and document all observations with screenshots and written notes. This mirrors the practical workflow used by SOC analysts and network security professionals when reviewing packet captures.

2. Objective

The project was carried out with the following objectives:

- Capture live network traffic on a Windows system using Wireshark.
- Identify the local network interface and IP configuration.
- Analyze Ethernet frames and MAC addresses.
- Identify source and destination IP addresses for observed traffic.
- Analyze ICMP traffic generated through ping activity.
- Identify DNS queries and their corresponding responses.
- Analyze TCP connection establishment and communication patterns.
- Examine TLS traffic associated with HTTPS connections.
- Identify active network endpoints communicating with the system.
- Analyze conversations between pairs of network hosts.
- Apply Wireshark display filters to isolate traffic of interest.
- Identify unusual or potentially suspicious network behavior.
- Document the full investigation with screenshots and a written technical report.

3. Tools Used

The investigation relied on the following operating system, software, and command-line utilities:

Category	Tool / Component
----------	------------------

Operating System	Windows 10 / Windows 11
Packet Capture & Analysis	Wireshark (latest stable version)
Capture Driver	Npcap (installed alongside Wireshark)
Network Configuration	ipconfig
Connectivity / ICMP Testing	ping
DNS Resolution Testing	nslookup
Route Path Testing	tracert
Active Connection Review	netstat -ano
Evidence File Format	.pcapng

Command-line utilities were used to generate traffic and to independently verify what Wireshark captured, allowing operating-system-level connection data to be cross-checked against packet-level evidence.

4. Network Environment

Before capturing traffic, the local network configuration was identified using the **ipconfig** command in Command Prompt. This established a baseline for interpreting source and destination addresses observed later in Wireshark.

```
ipconfig
```

Example recorded configuration:

Parameter	Example Value
Local IPv4 Address	192.168.5.121
Subnet Mask	255.255.255.0
Default Gateway	192.168.5.1
DNS Server	103.143.168.2
Network Interface	Wi-Fi / Ethernet

Within Wireshark, the corresponding capture interface was identified from the list of available adapters (e.g. Wi-Fi, Ethernet, Npcap Loopback Adapter) and selected based on which interface was actively carrying traffic to and from the system.

Recommended environment for reproducing this investigation:

Requirement	Specification
-------------	---------------

OS	Windows 10 / Windows 11
RAM	4 GB or higher
Processor	Intel / AMD
Network	Ethernet or Wi-Fi
Capture Software	Wireshark (latest stable) + Npcap

5. Packet Capture Method

Once the correct interface was selected in Wireshark, live capture was started and representative network activity was deliberately generated so that the capture would contain a meaningful mix of protocols for analysis.

5.1 Traffic Generation Activities

- **ICMP:** `ping 8.8.8.8` — generates Echo Request / Echo Reply packets.
- **DNS:** `nslookup google.com` and normal web browsing — generates DNS query/response traffic.
- **Traceroute:** `tracert 8.8.8.8` — reveals routing path and intermediate hops.
- **TCP / HTTPS:** visiting an HTTPS site such as `https://www.google.com` — generates TCP, TLS, HTTPS, and DNS traffic together.
- **Connection Verification:** `netstat -ano` — used to compare OS-level connection state with Wireshark's view.

5.2 Stopping and Saving the Capture

After sufficient traffic was generated, the capture was stopped in Wireshark and saved as the primary evidence file for the investigation:

```
PCAP/network_capture.pcapng
```

5.3 Packet Structure Reviewed

Each captured packet was reviewed layer by layer, following the general encapsulation model:

```
Frame
-> Ethernet
-> IP
-> TCP / UDP / ICMP
-> Application Protocol
```

6. ICMP Analysis

ICMP traffic generated by the `ping` command was isolated using the following Wireshark display filter:

```
icmp
```

Additional targeted filters used during the investigation:

Purpose	Filter
All ICMP traffic	icmp
Echo Request only	icmp.type == 8
Echo Reply only	icmp.type == 0

Example observed exchange:

```
Request -> Source: 192.168.5.121 Destination: 8.8.8.8 Type: Echo Request
Reply -> Source: 8.8.8.8 Destination: 192.168.5.121 Type: Echo Reply
```

For each exchange, the source/destination IP addresses, ICMP sequence numbers, and TTL values were recorded and matched between request and reply to confirm normal round-trip behavior. Supporting evidence was captured as Screenshots/icmp.png.

7. DNS Analysis

DNS traffic was isolated using:

```
dns
```

Query and response direction was distinguished using:

Purpose	Filter
All DNS traffic	dns
DNS queries only	dns.flags.response == 0
DNS responses only	dns.flags.response == 1
DNS over UDP (port 53)	udp.port == 53

For each DNS transaction, the query name, query type (A, AAAA, CNAME, MX, TXT), and the resolved IP address in the corresponding response were recorded. Example:

```
Query: google.com
Type: A
Response: IPv4 address returned by DNS server
```

This analysis helps establish which domains the system contacted during the capture window, which is useful both for troubleshooting and for baseline security monitoring. Evidence was captured as Screenshots/dns.png.

8. TCP Analysis

TCP traffic was isolated using:

```
tcp
```

Connection establishment was verified by locating the standard TCP three-way handshake:

```
Client -> Server : SYN
Server -> Client : SYN, ACK
Client -> Server : ACK
```

Additional filters used to examine TCP behavior in detail:

Purpose	Filter
SYN packets	<code>tcp.flags.syn == 1</code>
SYN without ACK (half-open attempts)	<code>tcp.flags.syn == 1 && tcp.flags.ack == 0</code>
TCP resets	<code>tcp.flags.reset == 1</code>
Retransmissions	<code>tcp.analysis.retransmission</code>
General TCP analysis flags/errors	<code>tcp.analysis.flags</code>

These filters allowed connection setup, teardown, and anomalies such as retransmissions or resets to be reviewed independently of normal data exchange. Evidence was captured as `Screenshots/tcp.png`.

9. TLS / HTTPS Analysis

Encrypted HTTPS traffic was isolated using either of the following filters:

```
tls
tcp.port == 443
```

The TLS handshake was inspected for the following elements:

- **Client Hello** — initiated by the local system, including supported TLS versions and cipher suites, and often the Server Name Indication (SNI).
- **Server Hello** — the server's chosen TLS version and cipher suite.
- **Certificate** — the server's presented certificate information.
- **Encrypted Application Data** — the encrypted payload exchanged after the handshake completes.

Because modern HTTPS traffic is encrypted end-to-end, the actual webpage content is not visible in the capture; only handshake metadata (TLS version, cipher suite, SNI, certificate details) can be observed in plaintext. This is expected and confirms that the connection is properly encrypted. Evidence was captured as `Screenshots/tls.png`.

10. Endpoint Analysis

Wireshark's built-in endpoint statistics were reviewed via **Statistics** → **Endpoints**, covering IPv4, IPv6, TCP, UDP, and Ethernet endpoint categories.

For each IPv4 endpoint, the following fields were recorded:

- Address
- Total Packets
- Total Bytes
- Tx Packets / Tx Bytes (transmitted)
- Rx Packets / Rx Bytes (received)

This view was used to identify which hosts generated the highest volume of traffic during the capture window, providing a quick way to spot dominant or unexpected communication partners. Evidence was captured as `Screenshots/endpoints.png`.

11. Conversation Analysis

Host-to-host communication was reviewed via **Statistics** → **Conversations**, across Ethernet, IPv4, IPv6, TCP, and UDP conversation types.

For each IP conversation pair, the following details were examined:

- Address A and Address B
- Packets A → B and Packets B → A
- Bytes A → B and Bytes B → A
- Conversation Duration

This helped establish clear communication relationships between the local system and each remote host, including how much data was exchanged and over what time span. Evidence was captured as `Screenshots/conversations.png`.

12. Suspicious Traffic Analysis

The goal of this stage was not to automatically label unusual packets as malicious, but to identify traffic patterns that deviate from expected behavior and warrant further review.

12.1 Repeated Connection Attempts

Indicators: multiple SYN packets, repeated connection failures, repeated destination attempts.

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

12.2 TCP Resets

```
tcp.flags.reset == 1
```

A high volume of resets may warrant further review depending on the application and network context.

12.3 ICMP Anomalies

Indicators: high-volume ICMP traffic, unexpected destinations, repeated requests, unusual packet sizes.

icmp

12.4 Suspicious DNS Activity

Indicators: a large number of DNS requests, repeated requests for unusual domains, unexpected external domains, or unusual query patterns.

dns

12.5 Unexpected External Connections

Endpoint and conversation views were reviewed for external addresses that were not expected in the test environment. An address was not treated as malicious solely because it was external; each case was evaluated in context before drawing any conclusion.

13. Findings

Based on the captured evidence, the following findings were recorded:

#	Finding
1	ICMP communication was observed between the local system and an external destination, with matching Echo Request and Echo Reply pairs.
2	DNS queries and their corresponding responses were observed for domain name resolution during normal browsing activity.
3	TCP connections were established correctly using the standard SYN / SYN-ACK / ACK three-way handshake.
4	TLS traffic was observed for encrypted HTTPS communication, with handshake metadata (Client Hello, Server Hello, Certificate) visible while application data remained encrypted.
5	Multiple network endpoints and host-to-host conversations were identified and quantified by packet and byte counts.
6	No traffic was found to meet a sufficient evidentiary threshold to be classified as malicious; any traffic flagged for review should be validated further using packet frequency, destination, protocol, timing, and application context.

14. Conclusion

The **Network Traffic Analysis Using Wireshark** investigation successfully demonstrated a complete, evidence-based packet analysis workflow: generating representative traffic, capturing it with Wireshark and Npcap, applying targeted display filters, inspecting individual protocol layers, and correlating findings with native Windows command-line tools.

Through this process, the investigation identified network hosts, IP and MAC addresses, DNS activity, ICMP communication, TCP connection behavior, and TLS/HTTPS encrypted traffic, and summarized host relationships using Wireshark's Endpoints and Conversations statistics. A structured review of potentially suspicious patterns — repeated connections, TCP resets, ICMP anomalies, and unusual DNS activity — was conducted without over-classifying normal traffic as malicious.

The resulting PCAP file, screenshots, written analysis notes, and this report together provide documented evidence of the investigation and establish a practical foundation for more advanced work in network security monitoring, SOC operations, threat detection, incident response, PCAP forensics, IDS/IPS analysis, and SIEM monitoring.

Disclaimer: This project is intended for educational, defensive, and authorized network-analysis purposes only. Only capture and analyze traffic on systems and networks where you have explicit permission to perform monitoring or testing.
